走行ロボット
V2

拡張

- Aボタンで、走行スタート
  - Bボタンで黒、Aボタンで、白とスタート
  -
- タイムアウト
  - lineTraceWtihTimer関数を追加
- 他段階走行
  - lineTraceUntilStop関数が大きく変更している
  - 使わなくても、準備だけしておける
- ラインを利用した回転（タイムアウト利用）
- ラインを利用したスポットアタック（タイムアウト利用）
- 自動キャリブレーション（タイムアウト利用）

Aボタンで、走行スタート

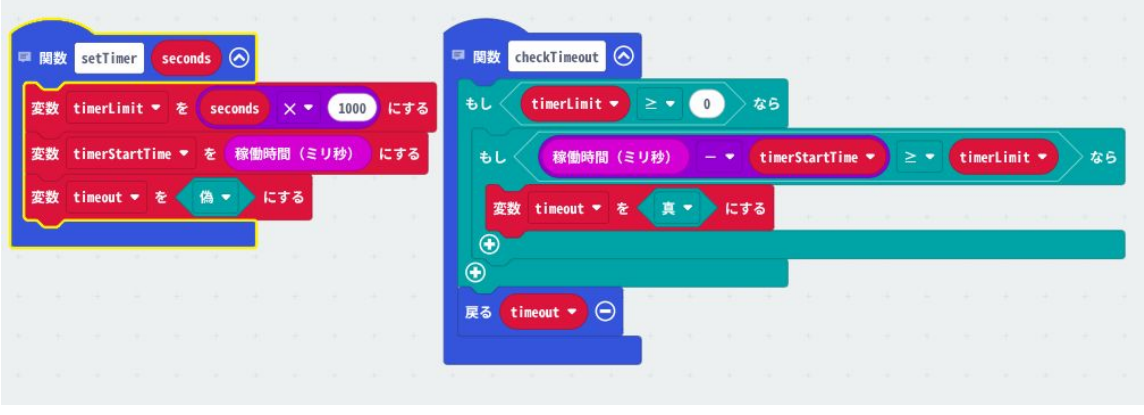
- 問題点
  - ロゴマークの反応が悪い時がある
- 解決策
  - Aボタンで、走行スタート
  - その場で、キャリブレーション
    - Aボタン：白とスタート
    - その他のボタンは、今までと同じ
  - USB接続で、黒を事前に測定し、変数に設定。Aボタンのみ操作
    - 本番で、暴走して電源を切ったら、そこで終わるので、普段から練習が必要
  - 自動キャリブレーション（autoCalibration関数を使う）安定していない
    - A+B：走行パターン指定
    - Aボタン：スタート（ロゴマークも利用可能）
    - スタート後に、プログラムにて白、黒をキャリブレーション



タイムアウト

- 問題点
  - スポット前にアイテムが密集して、停止ラインまで行けない。その場で、動けなくなり、走行が継続できない。
- 解決策
  - タイムアウトにより、停止ラインまで、行けなかったと判断して、次のアクションを取ることができる。
  - タイムアウト前に setTImerを呼ぶと、新しい値で、リセットされる。同時に使えない。

setTimer関数
checkTimeout関数
lineTraceUntilStop関数



タイムアウト付きライントレース

- maxTime:黒黒までのタイムアウト時間（秒）
  - 小数可能 0.5, 1.5
  - -1 タイムアウトなし
- maxHoldTIme:黒黒処理のタイムアウト時間（秒）
  - 小数可能 0.5, 1.5
  - -1 タイムアウトなし





タイムアウトライントレースの呼び出しと戻り（例）



タイムアウトの処理（黒黒まで　10秒経過）

タイムアウトの処理（黒黒処理中　2.5秒経過）

正常時の動作

10秒　　2.5秒



他段階走行

- 直進が続いたら
  - 始動時は、大きく
  - すぐに、少し小さく
  - 長く続いたら、定速
- 回転が続いたら
  - 小さく回転
  - スピン
  - 減速スピン
- 今までと同じように使う
  - 「もし」を利用しない

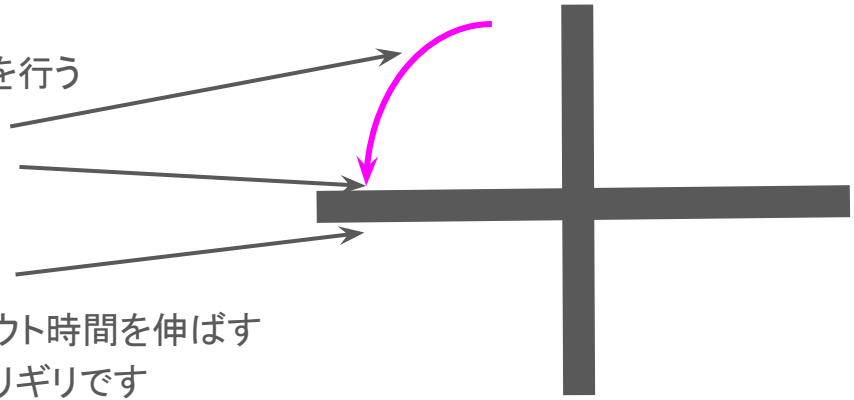




lineTraceUntilStop関数
goStraight関数
turnRight関数
turnLeft関数

ラインを利用した回転

- 問題
  - ロボットが斜めに停止すると、回転角度が変わる
- 解決策
  - 十字になっているラインを利用して 90度回転を行う
    - しばらく、回転
    - センサー値を見て、黒に変わるか確認
    - 黒に変わったら、白に変わるか確認
    - 白になったら、停止
  - ラインにたどり着く前に、止まるなら、タイムアウト時間を伸ばす
  - 設定されているタイムアウト時間は、かなりギリギリです



rotateLeftToLine関数:lines 横切るラインの数、タイムアウト時間(秒)
rotaleRgihtToLine関数:lines 横切るラインの数、タイムアウト時間(秒)

ラインを利用したスポットアタック

- 問題点
  - アタックから戻る時、正しいラインに戻れない
- 解決策
  - ラインを見ながらロボットの向きを大きく変えないで、アタックする
  - アタック後、一度バックして前進のライントレースを行い向きを補正する
  - 他にも方法があるかも！1つの方法として、提案。

attackSpot関数
back1関数
back2関数
lineTraceBack関数

どこを走行しているかを知る

- laneNumber: レーン番号
  - コースのどのレーンを走行しているか
  - 1から6
- laneMode: レーンモード
  - 0:通常
  - 1:アタック
  - 2:次のレーンへ移動
- スタートからの経過時間
  - 経過時間 - startTime
- タイマーの経過時間
  - 経過時間 - timerStartTime



自動キャリブレーション



停止線を横切る時にキャリブレーションする

- 白 -> 黒 -> 白へと変わる値を見る
- 最初は、白として確定
- 走行しながら、センサーの値を見て、前回の値より大きければ、黒として使う
- 最大のセンサー値より 100 より小さくなったら、黒を終了したとする
- しきい値を計算して、しきい値以下になったら、白と認識して終了
- 失敗したときは、プログラムで指定した既定値を使う
    - **既定値は、会場で事前に測定してプログラムに書いておく**

autoCalibration関数

関数名、変数を日本語にする

- 関数名
  - 関数[runPartten1] を関数[パターン１]に変更
- 変数名
  - 変数を右クリックでメニューを開く
  - 「変数の名前を変更」を選択
  - 日本語名を入力
- 引数名（関数に付いている変数）
  - 関数を右クリックでメニューを開く
  - 「関数の編集」を選択
  - 引数を日本語名に変更
  - 注意
    - 引数の値を変更する時は、変数と同じ名前にする





V2コードへ今のプログラムから移す方法

- 今のプログラムをファイルへ保存し、バックアップを取る
- MakeCodeをもう一つ開いて、QRコードから V2のコードを読み込む
- 今のプログラムを表示して、異なる箇所を見つける
- V2のコードを修正する

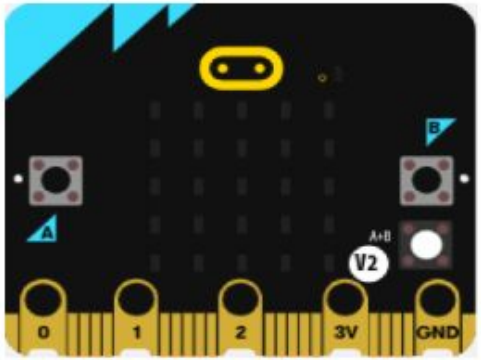
Ctrl+f で検索
JavaScriptでコピー＆ペーストをする方法もある

V2コードを今のプログラムにコピーする方法

- 今のプログラムをファイルへ保存し、バックアップを取る
  - 「JavaScript」をクリック
- MakeCodeをもう一つ開いて、QRコードからV2のコードを読み込む
  - 「JavaScript」をクリック
- コピーする箇所を選択し、Ctrl+c でコピー
  - `function` から　同じカラム位置の }まで
- 今のプログラムを表示して、一番先頭にCtrl+vで貼り付け
- 既存と同じ関数名の時は、エラーになる
  - 今までの関数名を変える
- 存在しない変数名は、エラーになるの
  - V2のコードの最後の方までスクロール
  - let 変数名 = 0　などの1行をコピーして、今のプログラムの一番先頭に Ctl+vで貼り付け
- 今のプログラムを表示して、「ブロック」をクリック

走行パターンV2 ライントレース キャリブレーションDF Driver

プロジェクトを共有

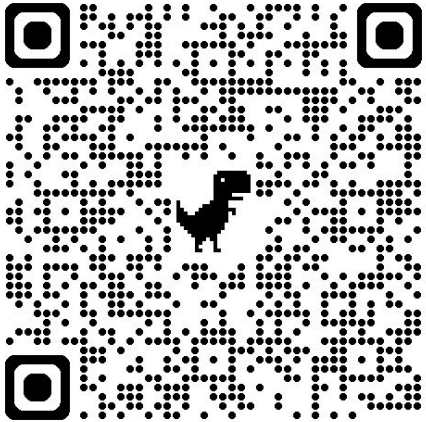


Project Link

https://makecode.microbit.org/S00921-48953-42665-11948

コピー

V2資料





パターン関連

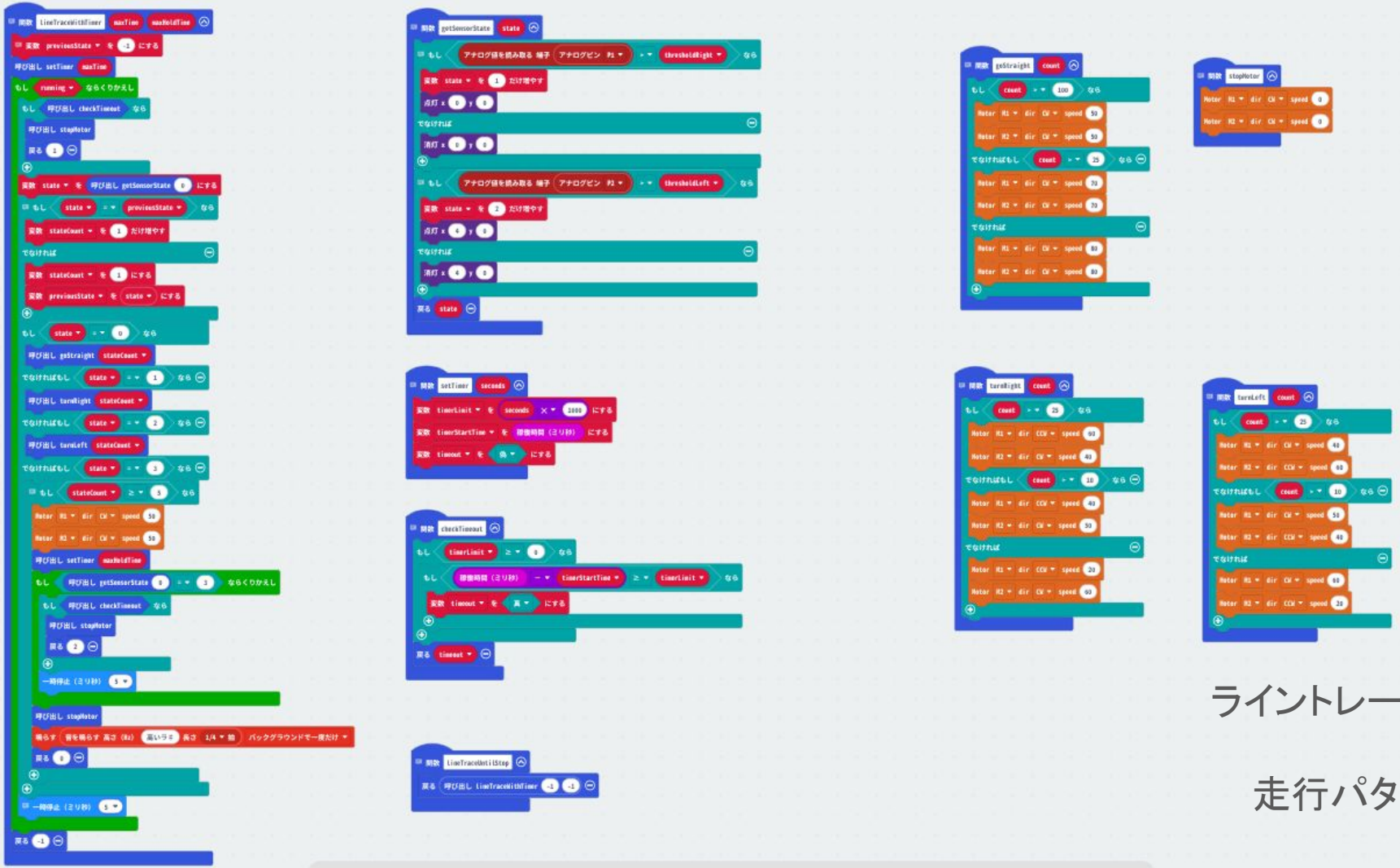
走行パターンV2

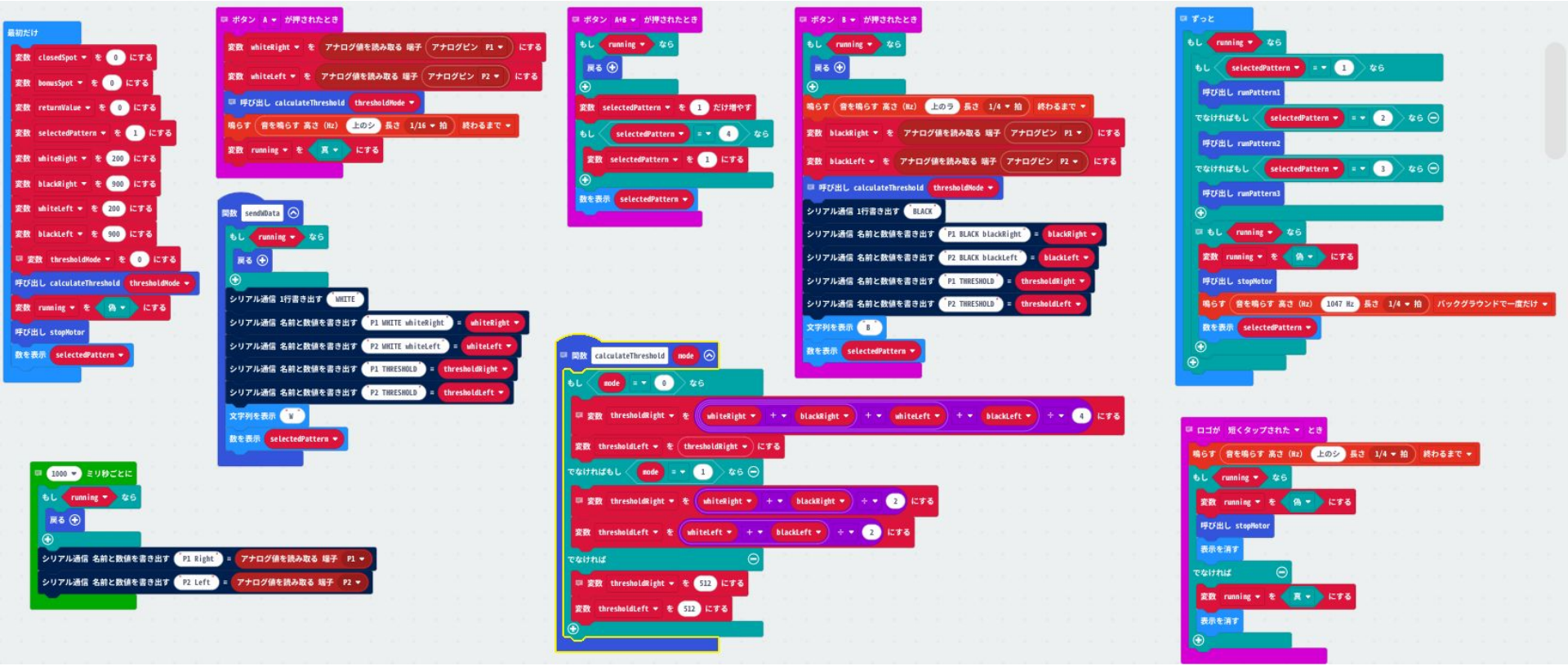




アタック関連

走行パターンV2


ライントレース関連
走行パターンV2



スタート関連

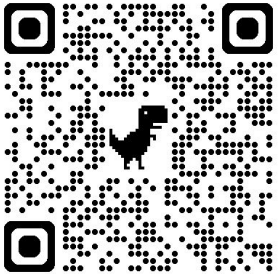
走行パターンV2

参照

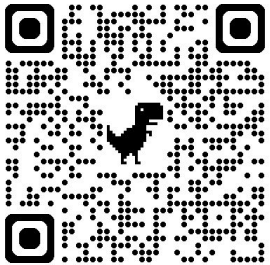
ストップウォッチ



V2資料



スコアー

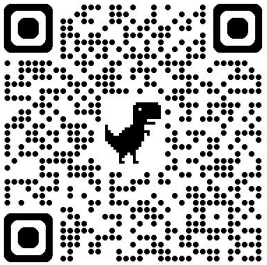


V2コード

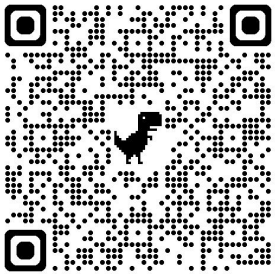


参照

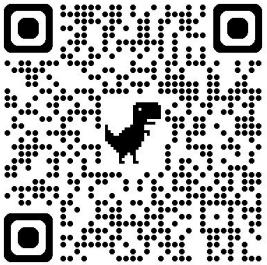
ストップウォッチ



V2資料



スコアー



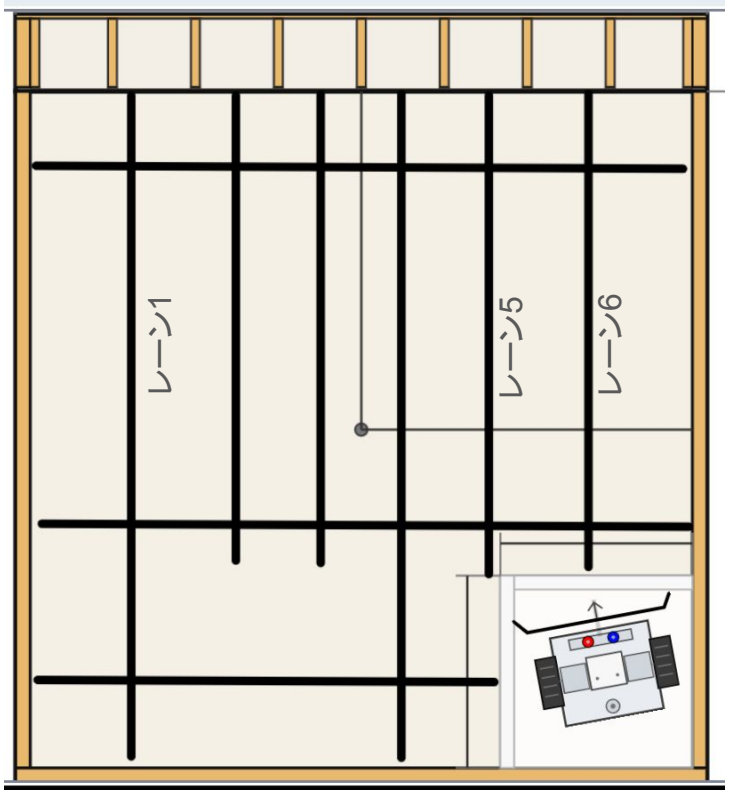
V2コード



レーン6,5から入ってレーン1へ移動

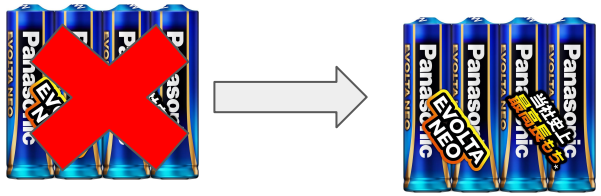
```
関数 runPattern3
変数 laneNumber を 7 にする
呼び出し lineTraceUntilStop
くりかえし 5 回
    変数 laneNumber を -1 だけ増やす
    呼び出し spotroundtrip
    呼び出し nextR
変数 laneNumber を -1 だけ増やす
呼び出し last
```

```
関数 nextR
変数 laneMode を 2 にする
呼び出し rotateRightToLine 1 1
呼び出し lineTraceUntilStop
呼び出し rotateRightToLine 1 1
変数 laneMode を 0 にする
```

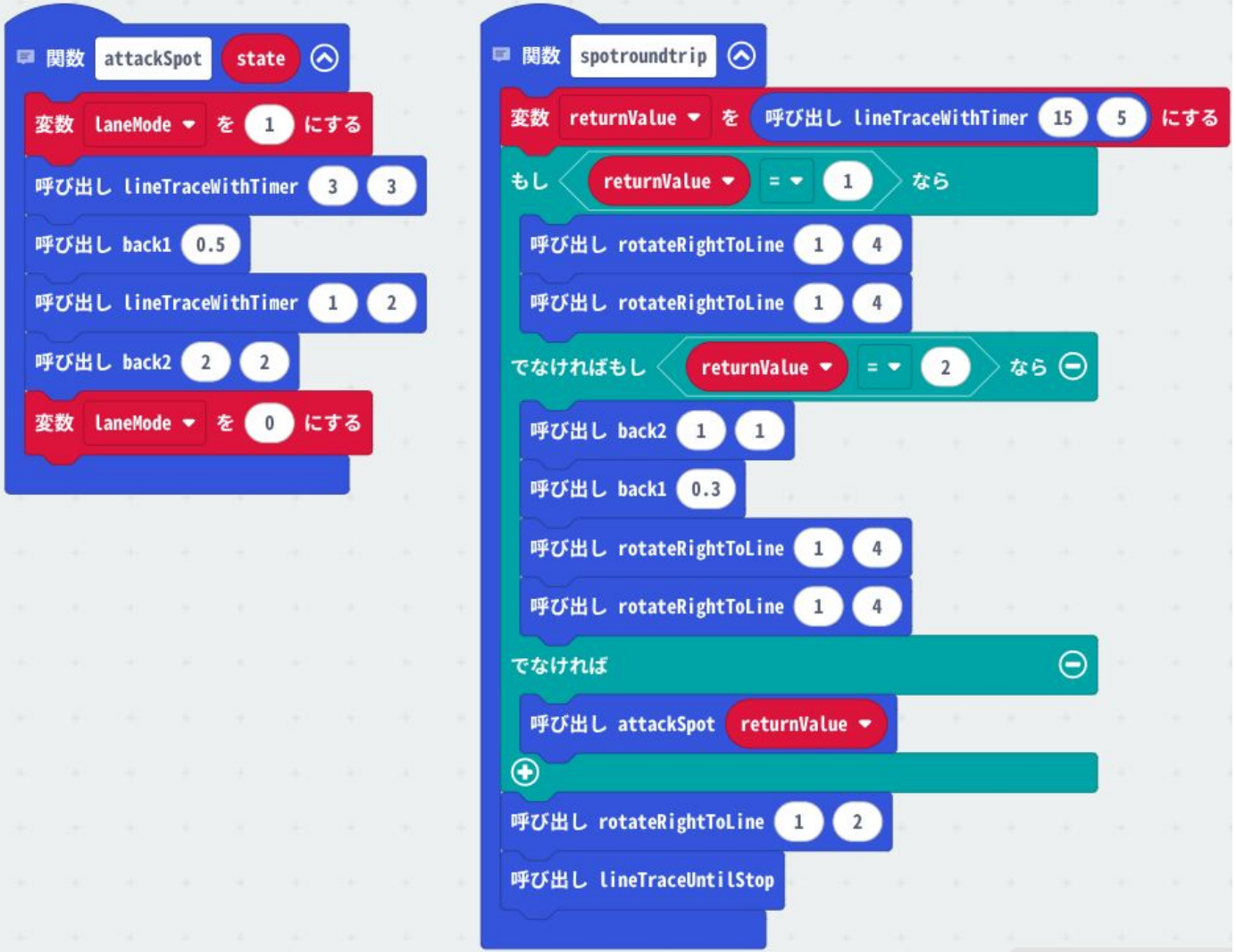


電池の管理

- 問題
  - 弱くなってる電池で、数値を決め
  - 競技で、新しい電池で、数値がうまく対応しない
- 解決策
  - 2,3組の電池を用意。練習で、ローテーションをして数値を決める。
  - 全体的に少し弱くなるが、交換するまで弱くならない。
  - 競技は、ローテーションしている電池を使う






関数 attackSpot state
変数 laneMode を 1 にする
呼び出し lineTraceWithTimer 3 3
呼び出し back1 0.5
呼び出し lineTraceWithTimer 1 2
呼び出し back2 2 2
変数 laneMode を 0 にする
関数 spotroundtrip
変数 returnValue を 呼び出し lineTraceWithTimer 15 5 にする
もし returnValue = 1 なら
呼び出し rotateRightToLine 1 4
呼び出し rotateRightToLine 1 4
でなければもし returnValue = 2 なら
呼び出し back2 1 1
呼び出し back1 0.3
呼び出し rotateRightToLine 1 4
呼び出し rotateRightToLine 1 4
でなければ
呼び出し attackSpot returnValue
呼び出し rotateRightToLine 1 2
呼び出し lineTraceUntilStop